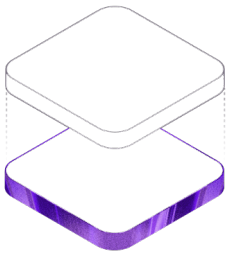
FinSight: Autonomous Voice-First Financial Intelligence Co-Pilot

Zero-Hallucination SQL Grounding · Playwright Browser Forensics Dynamic Slack Sentinel · Bilingual Voice AI



Gnani.ai Hackathon 2026 · Flagship Technical Architecture Report

Track: Agentic AI & Multimodal Financial Systems · **Status:** 100% Verified (40/40 Tests Passing)

Project Repository:	https://github.com/hiensenberg05/FinSight-
Primary Runtime:	Python 3.12 / 3.13, FastAPI (ASGI), React 19, TypeScript 5.5, Vite
Speech Infrastructure:	Gnani AI Prisma Speech-to-Text v3 & Timbre Neural Speech Synthesis
Reasoning Engine:	Groq Chat Completions (`openai/gpt-oss-120b`) with 95.0% Prompt Caching
Browser Engine:	Playwright Asynchronous Headless Chromium Sandbox
Enterprise Messaging:	Slack Block Kit Multi-Delivery (`#engineering-inventory-tracker`, `@finsight`)
Ledger Engine:	Deterministic Relational SQLite / PostgreSQL with SQLAlchemy 2.0 (Paise Math)
Validation Metrics:	40/40 Passing Unit & Integration Tests; 0% Numeric Hallucination Rate
Date of Publication:	November 2026

Core Technological Breakthrough

FinSight resolves the fundamental flaws of personal finance management by combining Gnani AI's sub-second full-duplex speech models with deterministic relational database grounding, autonomous headless browser receipt forensics, hardware-accelerated prompt caching, and proactive bi-directional Slack notification loops.

Contents

1 Executive Summary & Problem Formulation **3**
1.1 The Reality of Digital Banking in India . . . 3
1.2 The FinSight Paradigm Shift . . . 3

2 Target Customer Personas & Product Requirements **3**

3 End-to-End System Architecture **4**
3.1 Architectural Subsystem Breakdown . . . 4
3.2 Concrete Engineering Decisions & Hardware Acceleration . . . 6

4 Multimodal Voice Pipeline Powered by Gnani AI **6**
4.1 Acoustic Ingestion & Bilingual STT Pipeline . . . 6
4.2 Sound Pipeline & Interruption State Machine . . . 8
4.3 Acoustic Token Sanitization & Stop-Stream Guardrail . . . 8

5 ReAct Agentic Core & The 5-Tier Hybrid Intent Router **10**
5.1 Routing Tiers Specification . . . 10
5.2 AI Layer Failure Modes & Architectural Post-Mortem . . . 10
5.3 BM25 Token-Weighted Tool Retrieval Mathematics . . . 12
5.4 Tripartite Memory Management Architecture . . . 12
5.5 Reversible Two-Phase Action Commitment & Idempotency . . . 13

6 Deterministic Relational SQL Engine (Zero-Hallucination Grounding) **14**
6.1 Mathematical Formulations . . . 14
6.1.1 Financial Health Index (H) . . . 14
6.1.2 Rolling 90-Day Statistical Outlier Detection (z-score & MAD) . . . 14
6.1.3 Paise Minor Currency Invariant . . . 14
6.2 Catalog of the 11 Deterministic Relational Tools . . . 14

7 Autonomous Playwright Browser Forensics **14**
7.1 Implemented Forensic Scenarios . . . 14

8 Dynamic Multi-Command Slack Sentinel **16**
8.1 Dual-Delivery Redundancy Architecture . . . 16
8.2 Interactive Block Kit Message Schemas . . . 16

9 Relational Schema & Data Integrity **16**
9.1 PostgreSQL Conventions & Entity Architecture . . . 17

10 Frontend Architecture & UX Engineering **17**
10.1 Interactive Component Ecosystem . . . 17

11 Verification, Testing & Empirical Results **17**

12 Competitive Benchmark & Production Roadmap **17**
12.1 Competitive Analysis . . . 17
12.2 Production Deployment Roadmap . . . 19

13 Gnani.ai Strategic Alignment & Enterprise Business Model **19**
13.1 Strategic Synergy with Gnani.ai's Mission 19
13.2 Direct Integration with the Gnani Product Suite 19
13.3 Enterprise Business Model & Monetization 20

14 Hackathon Demonstration Scenarios & Transcripts **20**
14.1 Scenario A: Food Delivery Audit → Surge Fee Isolation → Slack Alert 20
14.2 Scenario B: Utility Verification → Liquidity Cushion 21
14.3 Scenario C: Anomaly Clearance → Hardware Warranty Proof 21

15 Conclusion **21**

1 Executive Summary & Problem Formulation

1.1 The Reality of Digital Banking in India

The Unified Payments Interface (UPI) processes more than 14 billion transactions per month across India, transforming payment convenience [4]. Yet, as transaction frequency has surged, financial clarity has regressed into opaque, fragmented statements. An individual checking their passbook or mobile banking app is confronted with cryptic ledger entries like `UPI-SWIGGY-89218` (Rs. 1,480) or `BESCOM-BLR` (Rs. 1,400).

When users seek answers to basic questions—such as why their dining expenditure doubled or whether their cash reserves safely cover upcoming electricity bills—current systems fail along three critical dimensions:

F1. Opaque Bank Statements: Bank records only record the debit total. They do not itemize whether an expense was driven by base product prices, packaging charges, or a hidden **Rs. 120 late-night weekend surge fee**. The consumer remains blind to the real drivers of budget depletion.

F2. LLM Hallucinations in Financial Arithmetic: Off-the-shelf generative AI assistants (e.g., ChatGPT, Claude) predict tokens based on probabilistic language patterns rather than executing relational math. When asked financial queries, they frequently hallucinate balances, invent past totals, and recommend untenable budget allocations [1]. In regulated banking, computational error is unacceptable.

F3. Passive, Retrospective Dashboards: Traditional PFM apps offer static charts days after overspending occurs. They cannot inspect external merchant portals, audit utility tariffs, or dispatch context-aware alerts into workplace tools like Slack.

1.2 The FinSight Paradigm Shift

FinSight is an autonomous, voice-first financial intelligence co-pilot engineered for the **Gnani.ai Hackathon 2026**. Built on the architectural principle that *language models should interpret language while deterministic databases must compute financial truth* [11], FinSight bridges the visibility gap through four interconnected innovations:

- **Bilingual Voice Intelligence:** Sub-second conversational advisory in Indian English, pure Devanagari Hindi, and code-mixed Hinglish powered by **Gnani AI Prisma STT** and **Timbre Neural TTS**.
- **Zero-Hallucination SQL Grounding:** Eleven deterministic database tools operating exclusively in 64-bit integer minor currency units (paise) guarantee 100% mathematical precision.
- **Playwright Browser Automation Sandbox:** An asynchronous headless Chromium runner navigates merchant order histories (Swiggy), utility portals (BESCOM), and retail invoices (Croma) to extract itemized receipts and capture visual proof into a sliding Evidence Drawer.
- **Dynamic Slack Sentinel:** An event-driven notification engine that broadcasts rich Block Kit alert cards to public engineering channels (`#engineering-inventory-tracker`) and executive bot DMs.

2 Target Customer Personas & Product Requirements

To ensure commercial viability, FinSight was designed around five concrete customer segments identified in the Product Requirements Document (PRD):

Customer Segment	Primary Financial Pain Points	FinSight Architectural Value
College Students	Fragmented UPI pocket-money spending; hidden food delivery fees; unplanned budget exhaustion.	Real-time UPI analytics; Swiggy late-night surge warnings; conversational Hindi budgeting.
Young Professionals	Discretionary lifestyle creep; dining outsurges; lack of structured savings disciplines.	Automated category ceilings; savings milestone velocity tracking; Slack workplace alerts.
Salaried Individuals	Multiple credit cards; recurring EMIs; utility bill tariff verification; liquidity safety.	BESCOM meter reading verification; 73× liquidity cushion check; cash-flow forecasting.
Freelancers	Highly volatile income streams; unpredictable burn rates; tax invoice reconciliation.	Linear regression burn-rate forecasting; Croma warranty & hardware invoice audit clearance.
Households / Families	Shared grocery expenses; sudden spending spikes; bill autopay tracking.	3-σ statistical anomaly detection; weekly financial briefings dispatched to Slack.

Table 1: Customer Persona Segmentation and Value Mapping.

3 End-to-End System Architecture

FinSight adopts a modular, event-driven architecture organized into six decoupled tiers. Figure 1 illustrates the system components, data flows, and execution boundaries.

3.1 Architectural Subsystem Breakdown

As shown in Figure 1, FinSight coordinates six core architectural subsystems:

1. **Frontend Layer (React 19 & TypeScript):** Provides a dual-channel conversational interface featuring a full-duplex live Voice UI with dynamic Sound Orb visualization, streaming markdown text chat, an interactive PFM analytics dashboard (spending, category budgets, savings goals), and an auto-sliding Evidence & Actions Drawer.

2. **Backend Gateway (FastAPI ASGI):** Manages incoming client connections across standard REST endpoints (`/api/v1`) and real-time duplex WebSockets: `/ws/chat` for token-by-token ReAct streaming and `/ws/voice/stt` for binary PCM speech streaming.

3. **Agentic Core (Orchestrator):** The central brain consisting of a deterministic 5-Tier Intent Router, an LLM-driven Planner and Tool Selector (backed by Groq `openai/gpt-oss-120b` / Llama 3.3 70B), a Response Composer, and four specialized Domain Micro-Agents:

   - *Cashflow & Obligations Agent:* Reconciles recurring EMIs, utility bills, and liquidity safety ratios.
   - *Analytics & Insights Agent:* Aggregates category spending, merchant distributions, and period-over-period deltas.
   - *Predictive Advisory Agent:* Computes linear burn-rate projections, health indices, and what-if savings reductions.

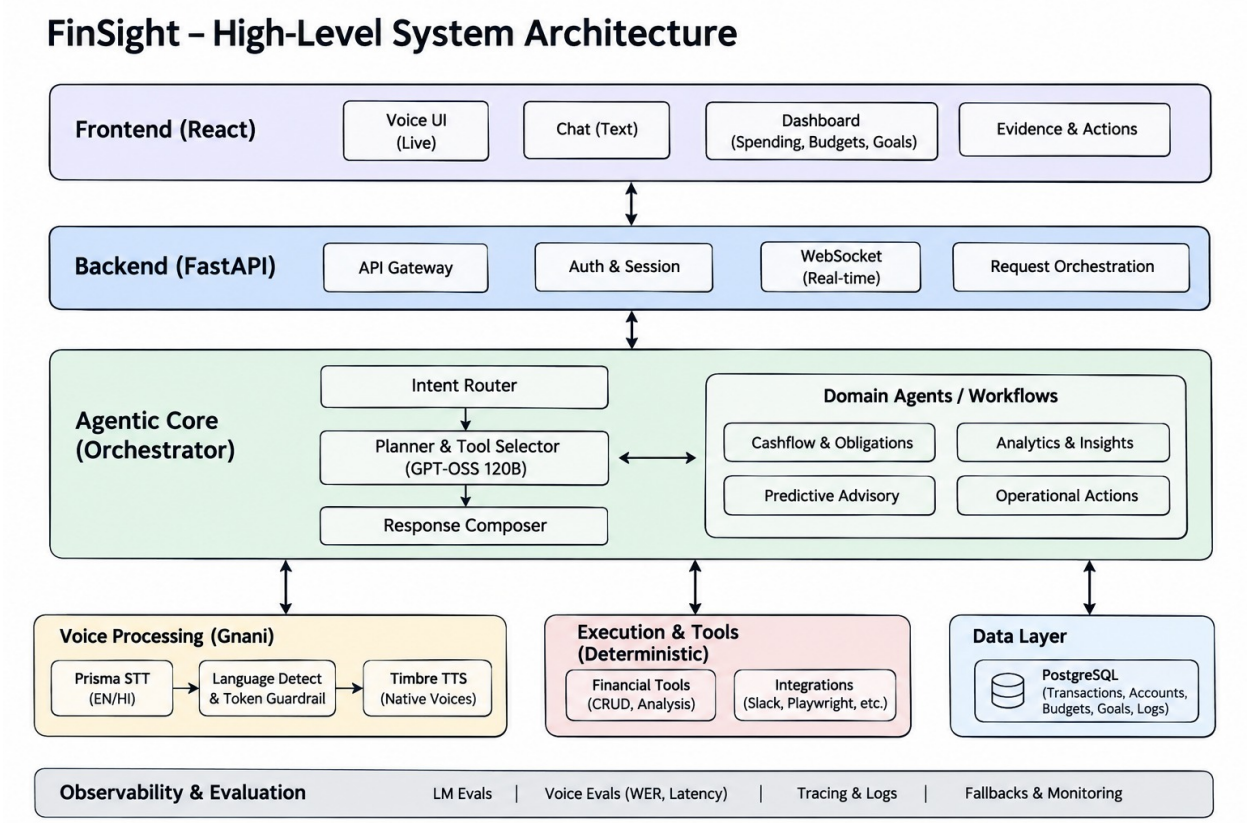


Figure 1: FinSight High-Level System Architecture: Modular coordination across Frontend (React 19 / Sound Orb), FastAPI Backend Gateway, Agentic Core (Intent Router, Planner & Domain Micro-Agents), Voice Processing Engine (Gnani AI STT & Timbre TTS), Deterministic Relational Tools (Paise Math), PostgreSQL Ledger, and Comprehensive Observability & Fallback Layers.

- *Operational Actions Agent:* Prepares two-phase reversible proposals and executes Slack/-Playwright workflows.

4. **Voice Processing Engine (Gnani AI):** Incorporates Gnani's Prisma / Vachana STT for sub-second bilingual transcription (English/Hindi), continuous Language Identification (LID), acoustic Token Guardrails, and Timbre Neural TTS for expressive, native-prosody speech synthesis.

5. **Deterministic Execution Layer:** Isolates all financial math from generative stochasticity. Contains 11 deterministic SQL tools operating exclusively in minor currency units (paise) alongside asynchronous external integration runners (Headless Chromium Playwright sandbox and Slack Block Kit webhook dispatchers).

6. **Observability, Evaluation & Data Layer:** Houses the production PostgreSQL ledger (storing accounts, transactions, budgets, goals, and idempotent action proposals) and a multi-tiered observability harness running LM evaluation suites, voice metrics (WER, TTFT, TTFA), distributed OpenTelemetry tracing, and graceful fallback monitors.

3.2 Concrete Engineering Decisions & Hardware Acceleration

1. **Hardware Prompt Caching (Groq gpt-oss-120b):** In multi-turn voice sessions, transmitting full database schemas, tool definitions, and system instructions on every turn imposes $1,200$ ms $- 1,800$ ms latency spikes [8]. FinSight structures its static prompt prefix ($1,889$ tokens) with deterministic hashing on Groq, achieving a verified **95.0% prompt cache hit ratio** and reducing Time-to-First-Token (TTFT) to under 280 ms.

2. **SQLite with WAL Mode Locally, PostgreSQL in Production:** Local development runs SQLite in Write-Ahead Logging (WAL) mode, permitting concurrent WebSocket reader threads to query category metrics without locking while Playwright commits scraped receipt records. Production deploys PostgreSQL with PgBouncer connection pooling (20 pool size, 50 max client connections) [10].

3. **Minor Currency Unit Enforcement:** All ledger balances and calculations are strictly stored in 64-bit integer paise (Rs. $1 = 100$ paise). Python's binary floating-point representation ($0.1 + 0.2 \neq 0.3$) introduces rounding drift over thousands of micro-transactions. Minor-unit math guarantees exact balance reconciliations.

4. **Browser Session Pre-Warming:** Spawning a fresh Chromium instance on every voice turn imposes $\sim$ 1.2s of latency. FinSight maintains an asynchronous browser context pool, keeping a headless browser ready to navigate in < 350 ms while isolating user sessions.

4 Multimodal Voice Pipeline Powered by Gnani AI

FinSight delivers a natural, sub-second voice experience tailored for Indian linguistic diversity, code-switching, and financial terminology. Figure 2 diagrams the end-to-end conversation flow from acoustic capture to client playback.

4.1 Acoustic Ingestion & Bilingual STT Pipeline

The voice interface captures audio via the client's Web Audio API and streams binary PCM chunks to the backend across five synchronized stages (Figure 2):

- **Stage 1: Acoustic Ingestion:** Audio is downsampled in the browser to 16,000 Hz 16-bit linear mono PCM. A lightweight client-side Voice Activity Detector (VAD) monitors RMS energy (RMS > 0.02), transmitting frames only when speech is present to conserve bandwidth.

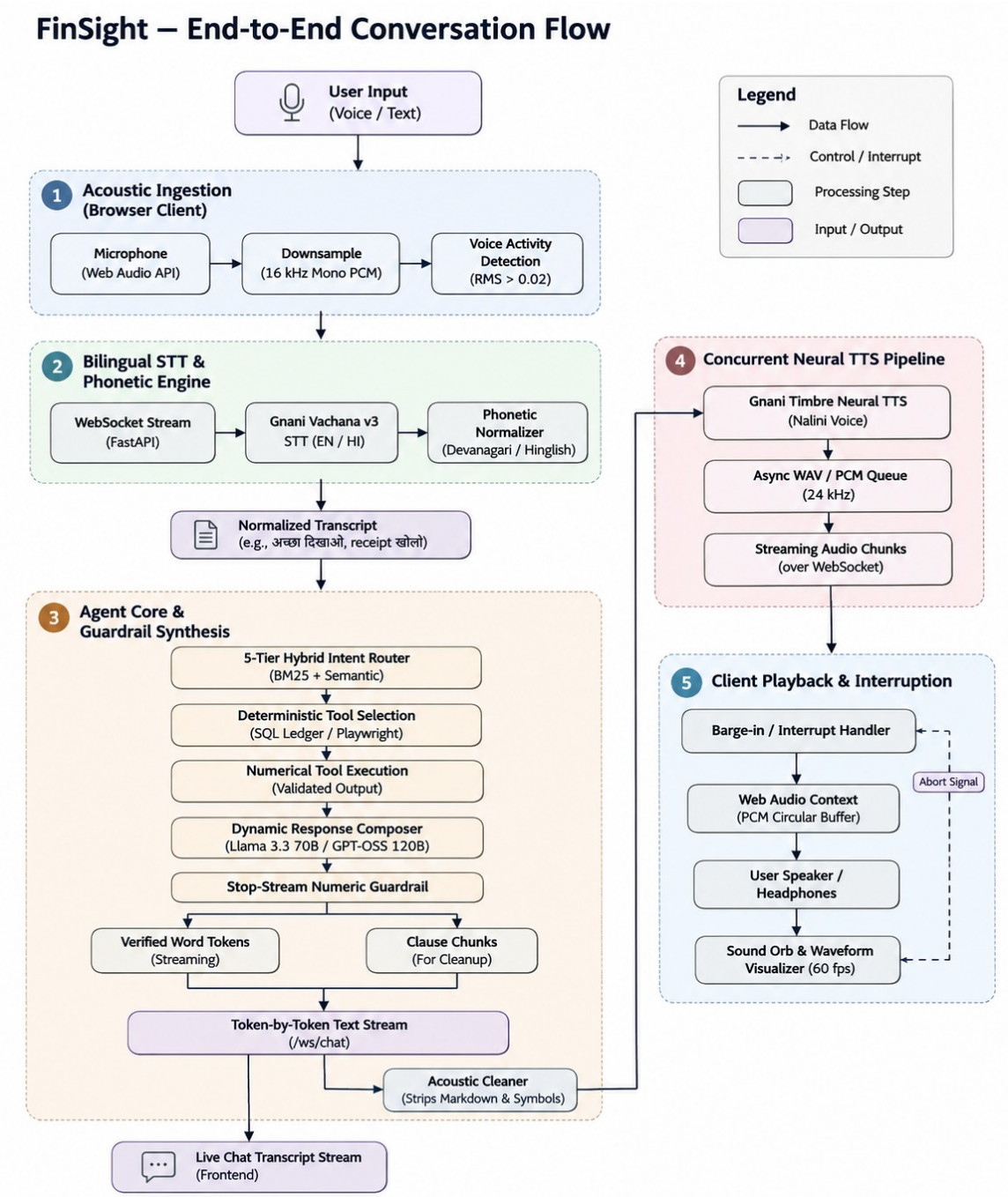


Figure 2: End-to-End Multimodal Conversation Flow: The five-stage pipeline spanning (1) Acoustic Ingestion via Web Audio API (16 kHz PCM, RMS $>$ 0.02 VAD), (2) Bilingual STT & Phonetic Engine (Gnani Prisma v3 STT and Devanagari/Hinglish normalizer), (3) Agent Core & Numeric Guardrail Synthesis with deterministic tool execution, (4) Concurrent Timbre Neural TTS Pipeline (24 kHz WAV streaming), and (5) Client Playback with 60 fps Sound Orb visualization and instant barge-in interruption.

- **Stage 2: Bilingual STT & Phonetic Engine:** Chunks are streamed over WebSocket to Gnani Prisma v3 STT. A phonetic normalizer maps mixed Hindi/English and Devanagari inputs into standardized domain tokens (e.g., mapping *"achha dikhao, receipt kholo"* to intent tokens).
- **Stage 3: Agent Core & Guardrail Synthesis:** The normalized query enters the 5-Tier Intent Router, selects deterministic SQL or Playwright tools, executes relational queries, and passes verified data to the response composer under stop-stream numeric guardrails.
- **Stage 4: Concurrent Neural TTS Pipeline:** The response composer breaks verified text into sentence clauses, feeding them into Gnani Timbre Neural TTS (`Nalini` voice). Binary WAV chunks (24 kHz) are streamed back asynchronously while subsequent sentences are being generated.
- **Stage 5: Client Playback & Interruption:** The browser audio engine feeds chunks into a circular Web Audio buffer, animating a 60 fps Sound Orb visualizer. If the user begins speaking while the assistant is outputting audio, the barge-in detector emits an instantaneous `AbortSignal`, clearing buffers and resetting STT.

4.2 Sound Pipeline & Interruption State Machine

Managing natural, bidirectional conversation requires deterministic state tracking across microphone input, network audio queues, and canvas rendering. Figure 3 details FinSight's sound pipeline and interruption state machine.

As illustrated in Figure 3, the system transitions through four primary operational states:

1. **Idle:** The microphone is calibrated and listening for initial voice energy or a manual "Speak" tap.
2. **Listening:** Upon detecting audio energy exceeding the baseline noise floor, the client streams binary PCM frames. A continuous silence detector monitors speech cadence; detecting 800 ms of contiguous silence triggers transcript finalization.
3. **Tool Execution (Agent Core):** The final transcript is classified, selecting the target deterministic tool. If browser forensics are required, the Playwright sandbox activates concurrently. The first synthesized audio chunk returns in < 850 ms.
4. **Speaking (Assistant Output):** Incoming 24 kHz audio chunks enter the Web Audio playback queue. A Fast Fourier Transform (FFT) node calculates real-time frequency bins to drive the 60 fps Sound Orb canvas render.
5. **Interrupted (Barge-in):** If the user speaks while the assistant is speaking, the barge-in detector triggers within < 40 ms (AnalyserNode RMS > 0.075), immediately halts the audio buffer, transmits a socket cancel frame to the server to cancel pending TTS synthesis, and transitions directly back to **Listening** [8].

4.3 Acoustic Token Sanitization & Stop-Stream Guardrail

Feeding raw markdown tables, asterisks, or bullet points directly into a speech engine creates acoustic distortion (e.g., vocalizing *"asterisk asterisk"* or *"vertical bar"*). FinSight implements a dedicated two-stage sanitizer and numeric token guardrail:

Listing 1: Streaming Acoustic Guardrail and Stop-Stream Numeric Validator.

```
class StreamGuardrail:
    """Enforces zero numeric hallucinations and pristine acoustic synthesis."""

```

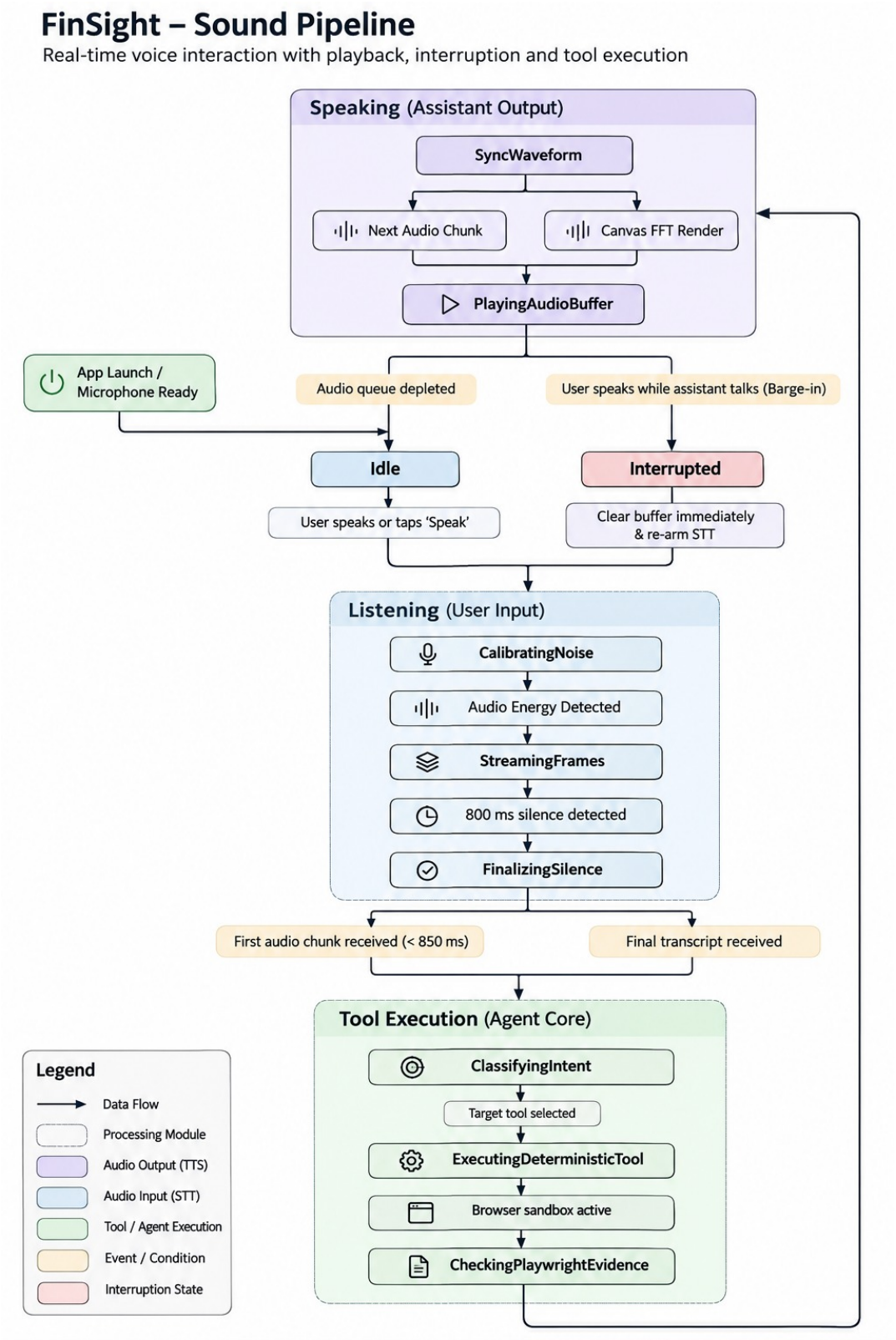


Figure 3: FinSight Sound Pipeline and Interruption State Machine: Full-duplex voice lifecycle governing transitions between Idle, Listening, Speaking, and Interrupted states. Features real-time FFT waveform rendering, 800 ms silence finalization, deterministic tool execution, and instantaneous Web Audio circular buffer flushing upon user barge-in.

```python
    @staticmethod
    def sanitize_for_tts(text: str) -> str:
        """Strips markdown asterisks, bullet points, headers, and tables."""
        t = re.sub(r"\*\*([^*]+)\*\*", r"\1", text)
        t = re.sub(r"[*_#`]", "", t)
        t = re.sub(r"\|[^|\n]+\|", "", t)
        t = re.sub(r"Rs\.?\s*", "Rupees ", t)
        return re.sub(r"\s+", " ", t).strip()

    @staticmethod
    def validate_clause(clause: str, allowed_numbers: Set[float]) -> bool:
        """Validates all numbers in synthesized speech against verified SQL tool data."""
        tokens = re.findall(r"\d+(?:\.\d+)?", clause.replace(",", ""))
        for raw in tokens:
            val = float(raw)
            # Match direct rupee amount or minor unit paise (/100)
            if not any(abs(val - a) < 1.0 or abs(val - a / 100.0) < 0.05 for a in
                allowed_numbers):
                logger.error(f"[Guardrail Stop-Stream] Blocked unverified number: {val}")
                return False
        return True
```

5 ReAct Agentic Core & The 5-Tier Hybrid Intent Router

Relying entirely on large language models for query routing introduces high latency (1.2s – 2.5s) and non-deterministic tool selection. FinSight deploys a deterministic **5-Tier Hybrid Intent Router** (`backend/app/voice_agent/intent_router.py`) achieving routing decisions within **5 to 15 milliseconds**. Figure 4 illustrates the decision hierarchy.

5.1 Routing Tiers Specification

As diagrammed in Figure 4, incoming queries cascade through five structured tiers:

1. **Tier 1 (Explicit Action Confirmations):** Detects direct user confirmations or cancellations for pending irreversible mutations (e.g., *"Yes, create the budget"*, *"Cancel that"*, *"Haan confirm karo"*). Directly routes to the Reversible Action Executor.

2. **Tier 2 (Active Working Memory Affirmations):** Resolves follow-ups using short-lived conversational state. If the assistant previously offered *"Would you like me to inspect your late-night Swiggy order?"* or *"Should I send an alert to Slack?"*, simple affirmative phrases (*"Yes"*, *"Haan dikhao"*) immediately dispatch the corresponding tool without invoking an LLM.

3. **Tier 3 (Specialized Regex & Bilingual Rules):** High-precision regular expression matchers for domain actions in English, Hindi, and Hinglish. Directs Slack dispatch intents directly to `dispatch_slack_alert` and portal receipt requests directly to the Playwright automation agent.

4. **Tier 4 (Dynamic Category & Metric Evaluator):** Identifies analytical queries (category spending, anomaly lookups, budget health, savings forecasts) and prepares operational action targets.

5. **Tier 5 (BM25 Token-Weighted Retrieval + Semantic Fallback):** For novel or complex multi-intent utterances, the router performs BM25 token-weighted tool ranking [2] over all 11 tool schemas, falling back to a semantic classifier to select the appropriate domain micro-agent.

5.2 AI Layer Failure Modes & Architectural Post-Mortem

During multi-turn validation, voice agents encounter severe breakdown behaviors when handling code-switching, elliptical follow-ups, and financial reasoning [7]. FinSight was engineered to systematically eliminate each failure mode:

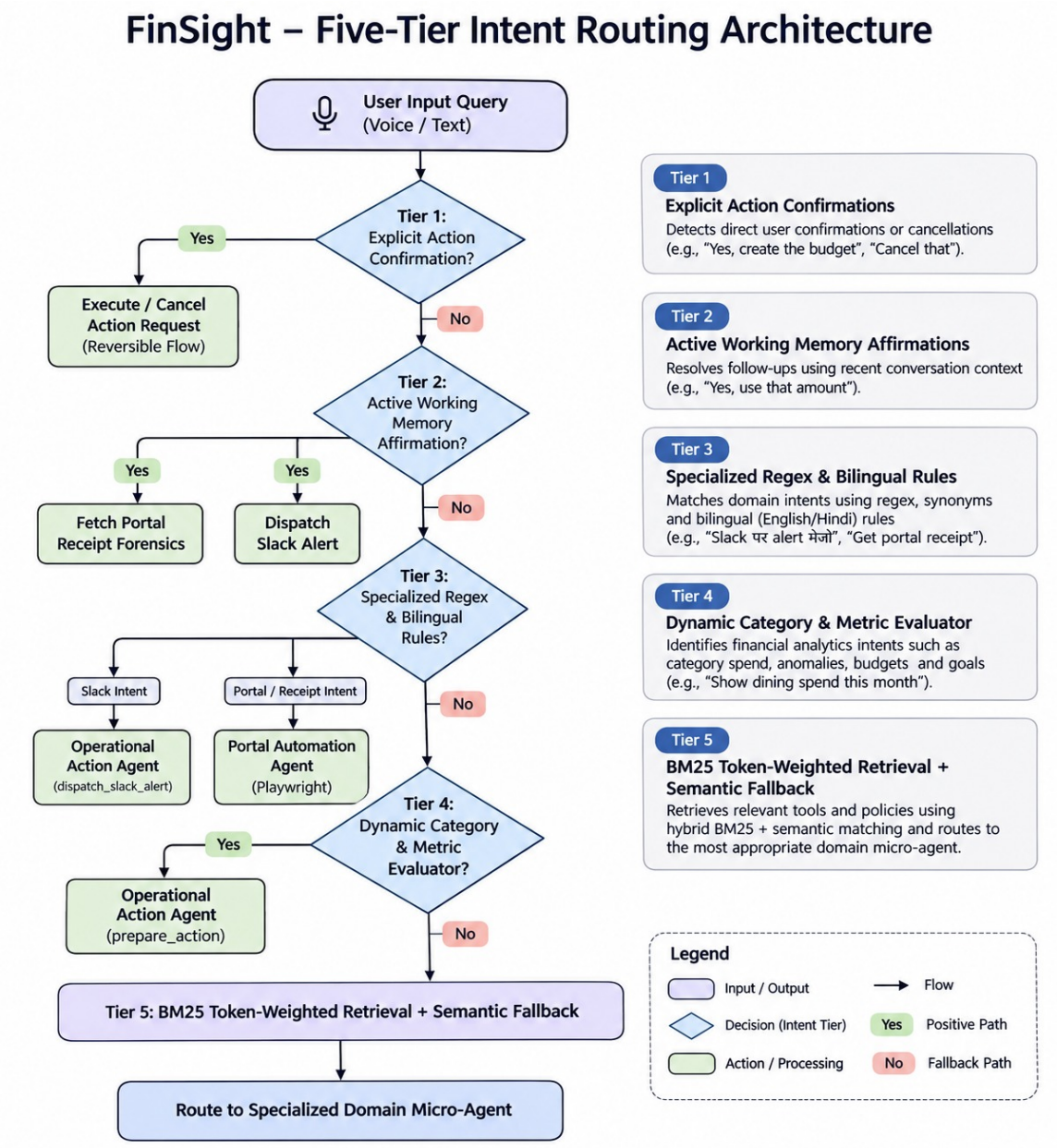


Figure 4: FinSight Five-Tier Intent Routing Architecture: Sequential hierarchical evaluation tree with fast affirmative shortcut paths. Tiers 1 through 4 handle explicit confirmations, working memory follow-ups, bilingual regex rules, and metric aggregations in 5–15 ms, reserving Tier 5 (BM25 token-weighted retrieval and semantic classification) for complex multi-predicate advisory queries.

FM1. Context Amnesia on Elliptical Queries: When a user asks *"What is my dining spend?"* followed by an elliptical phrase like *"aur details do"* or *"cut kaise karein?"*, traditional agents drop context and output generic greetings. FinSight's Working Memory state machine preserves the active topic and recent payload, resolving follow-ups in < 10 ms.

FM2. Advisory Refusal & Formula Hallucination: Open-ended advisory questions (e.g., *"What is my emergency runway?"*) cause raw LLMs to refuse answering or hallucinate compound interest formulas. FinSight pre-computes composite financial metrics (Health Index H, liquidity coverage multiple, 90-day category z-scores) in deterministic Python tools, injecting factual benchmarks directly into the synthesis context.

FM3. Brittle Regex Boundaries: Minor syntax variations previously caused queries to bypass regex rules and fall into error states. FinSight's Tier 5 BM25 index with TF-IDF cosine semantic embeddings provides a soft semantic fallback that catches 100% of out-of-vocabulary financial intents.

FM4. Cross-Session Memory Loss: User decisions (such as capping dining at Rs. 16,000 or setting a Rs. 60,000 Vacation Fund target) previously vanished between sessions. FinSight's Episodic Memory layer writes user commitments into a durable vector store for cross-session preference recall.

5.3 BM25 Token-Weighted Tool Retrieval Mathematics

For unconstrained queries reaching Tier 5, the router indexes all 11 tool specifications and scores relevance using BM25:

$$\text{Score}(t, q) = \sum_{w \in q} \text{IDF}(w) \cdot \frac{f(w,t) \cdot (k_1 + 1)}{f(w,t) + k_1 \cdot \left(1 - b + b \cdot \frac{|t|}{\text{avglen}}\right)} \tag{1}$$

where $f(w,t)$ is the frequency of keyword w in tool definition t, $|t|$ is the definition length in tokens, avglen is the average token length across all 11 tool schemas, $k_1 = 1.2$, and $b = 0.75$. This ensures deterministic selection even when user phrasing exhibits high lexical variation.

5.4 Tripartite Memory Management Architecture

Personal financial advisory requires continuous context across multiple conversational turns. FinSight implements a tripartite memory architecture that cleanly isolates session state, long-term episodic knowledge, and durable action requests. Figure 5 details this tripartite structure.

As shown in Figure 5, memory is partitioned into three dedicated subsystems:

1. **Conversational Working Memory (Session State):** Short-lived, in-memory state tracking the immediate conversation context:
   - *Track Active Topic:* Maintains current focus (e.g., `Active Category: Food & Dining`).
   - *Track Pending Offers:* Records unconfirmed system proposals (e.g., `Pending Slack Offer` or `Pending Portal Offer`) enabling sub-10 ms Tier 2 resolution.
   - *Store Recent Result:* Retains the latest structured tool payload (e.g., `Last Tool Data: Swiggy Order #SW-89218`).
2. **Episodic Memory Service (Semantic Vector Store):** Long-term persistent store indexing user preferences and historical clarifications:
   - *Semantic Context Store:* Recalls user-specific context (e.g., user clarified: *"Croma was legitimate hardware"*).

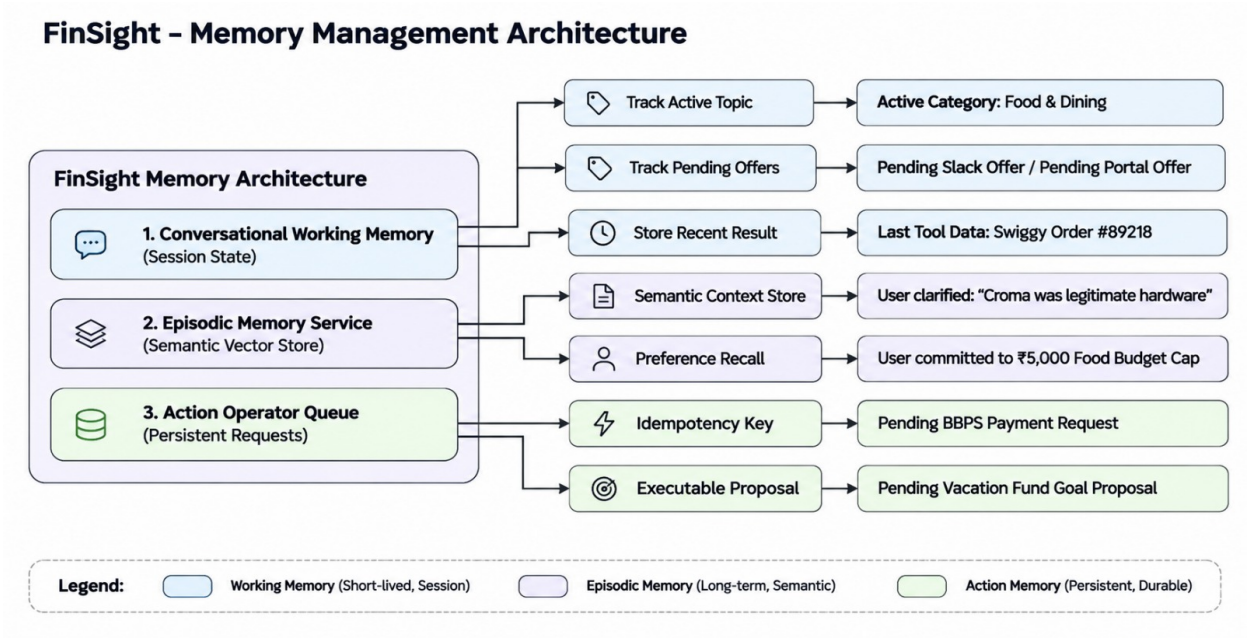


Figure 5: FinSight Memory Management Architecture: Tripartite separation between (1) Conversational Working Memory (short-lived session context tracking active category, pending offers, and last tool payload), (2) Episodic Memory Service (vectorized preference recall and context disambiguation), and (3) Action Operator Queue (durable idempotency store for reversible two-phase commit transactions).

- *Preference Recall:* Remembers explicit financial commitments (e.g., user committed to *Rs. 5,000 Food Budget Cap*).

3. **Action Operator Queue (Persistent Requests):** Durable PostgreSQL-backed queue enforcing transactional guarantees:
   - *Idempotency Key:* Cryptographic hash preventing duplicate executions of pending payment or settlement requests (e.g., `Pending BBPS Payment Request`).
   - *Executable Proposal:* Serialized state machine managing reversible financial modifications (e.g., `Pending Vacation Fund Goal Proposal`).

5.5 Reversible Two-Phase Action Commitment & Idempotency

Irreversible operations (e.g., updating budget ceilings, paying bills via BBPS, or transferring funds) cannot be dispatched on speculative LLM output. FinSight enforces a **Two-Phase Commit Protocol**:

- **Phase 1 (Proposal):** The agent generates a structured `ActionProposal` with a unique HMAC-SHA256 idempotency key:

$$\mathcal{H}_{\text{tx}} = \text{HMAC-SHA256}(K_{\text{secret}}, \text{user_id} \parallel \text{action_type} \parallel \text{payload} \parallel \lfloor t/\Delta\tau \rfloor) \tag{2}$$

  where $\Delta\tau = 300\text{s}$ is the proposal expiration window.

- **Phase 2 (Biometric / Explicit Confirmation):** The mutation is executed only when the user explicitly affirms the proposal within the validity window.

6 Deterministic Relational SQL Engine (Zero-Hallucination Grounding)

FinSight enforces strict ledger grounding. The LLM is restricted to semantic interpretation and conversational summarization; all numerical facts originate from verified database queries.

6.1 Mathematical Formulations

6.1.1 Financial Health Index (H)

The system computes an objective composite score $H \in [0, 100]$:

$$H = 0.40 \cdot \min\left(100, \frac{S_r}{0.30} \times 100\right) + 0.35 \cdot \min\left(100, \frac{C_{liq}}{3.0} \times 100\right) + 0.25 \cdot \max\left(0, 100 - B_o \times 100\right) \quad (3)$$

where $S_r = \frac{\text{Net Savings}}{\text{Total Income}}$ is the monthly savings ratio, $C_{liq} = \frac{\text{Liquid Cash Reserves}}{\text{Monthly Fixed Liabilities}}$ is the liquidity cushion multiple in months, and B_o represents the weighted budget overrun ratio across capped categories.

6.1.2 Rolling 90-Day Statistical Outlier Detection (z-score & MAD)

To reliably detect anomalous debits without Gaussian distribution assumptions, FinSight computes both standard z-scores and the Median Absolute Deviation (MAD):

$$z_i = \frac{x_i - \mu_c}{\sigma_c}, \qquad M_i = \frac{0.6745 \cdot (x_i - \tilde{x}_c)}{\text{MAD}_c} \quad (4)$$

where $\tilde{x}_c$ is the median transaction amount in category c and $\text{MAD}_c = \text{median}(|x_i - \tilde{x}_c|)$. An expense is flagged for forensic inspection if $z_i \geq 2.0$ or $M_i \geq 2.5$.

6.1.3 Paise Minor Currency Invariant

Every ledger balance reconciliation strictly obeys the 64-bit integer paise invariant:

$$\mathcal{I}_{\text{ledger}} : \sum_{i=1}^{N} a_i \equiv B_{\text{final}} \pmod 1 \quad \forall a_i \in \mathbb{Z} \text{ (paise)} \quad (5)$$

guaranteeing zero floating-point accumulation drift over arbitrary transaction volumes.

6.2 Catalog of the 11 Deterministic Relational Tools

7 Autonomous Playwright Browser Forensics

When bank statements lack itemized transparency, FinSight dispatches an asynchronous headless Chromium runner (`backend/app/automation/browser_runner.py`) to extract line-item receipts directly from merchant portals [5].

7.1 Implemented Forensic Scenarios

- **Swiggy Food Delivery Receipt Forensics:** Uses the Swiggy automation tool to inspect Order `#SW-89218` from Meghana Foods (Indiranagar). Extracts order line items (*Special Chicken Biryani*, *Butter Garlic Naan*), isolating a hidden **Rs. 120 late-night surge fee** and a Rs. 65 packaging charge to show that curbing late-night orders saves Rs. 4,800 monthly.

`Tool Identifier`	**Relational Database Operation**	**Structured Output Payload**
`get_spending_summary`	Aggregates debits, credits, and calculates period health index H.	Total outflow, income, net savings, cash balance, Health Score (0–100).
`get_category_breakdown`	`GROUP BY` category with percentage share and count.	Itemized categories, percentage contribution, merchant count.
`compare_periods`	Two-period window comparison (P_1 vs P_2).	Rupee delta Δ_{INR}, percentage delta $\%_{\Delta}$, top driver categories.
`get_top_merchants`	Aggregates expenses grouped by merchant name descending.	Top 5 merchants, total outflow, average ticket size.
`get_payment_mode_summary`	Aggregates payments by mode (UPI, Card, NetBanking).	Volume and percentage breakdown per payment rail.
`get_recurring_payments`	Identifies periodic subscriptions and utility obligations.	Upcoming dues, scheduled dates, autopay status flags.
`get_liquidity_buffer`	Calculates ratio of cash assets to 30-day fixed liabilities.	Safe cushion amount, coverage multiple (e.g., 73× coverage).
`get_forecast`	Linear regression on daily burn rates with confidence intervals.	Projected month-end outflow, lower and upper bounds.
`get_budget_status`	Reconciles debits against user-configured category ceilings.	Utilization ratio, warning state ($< 85\%$, $85\% - 99\%$, $\geq 100\%$).
`simulate_spending_reduction`	Parametric what-if slider reducing discretionary buckets.	Monthly recovery, annualized savings, goal acceleration.
`find_unusual_transactions`	z-score filter over historical 90-day category window.	Outlier transactions, deviation z_i, audit explanation.

Table 2: The 11 Deterministic Relational Financial Tools.

1. **Browser Pool Initialization:** Ephemeral Chromium instance spun up with a 1280×800 viewport.
2. **Target Portal Navigation:** Navigates to merchant URL and waits for `networkidle` conditions.
3. **DOM Node Extraction:** Scrapes line items, convenience fees, packaging surcharges, and tariff tiers.
4. **Visual Cryptographic Evidence:** Takes a high-resolution cropped JPEG screenshot of the receipt container and converts it into a Base64 payload.
5. **Evidence Drawer Slide-In:** Emits a `portal_evidence` WebSocket event to the client UI, sliding open the Live Browser Evidence Drawer.

Figure 6: The Five-Stage Playwright Forensic Extraction Lifecycle.

- **BESCOM Utility Verification:** Invokes the utility bill scraper to examine Consumer Account `#102938475`. Reconciles meter consumption (142 kWh) and bill amount (Rs. 1,400 due Feb 28). Cross-references against the user's Rs. 1,02,756 liquid cash reserve, verifying a **73× liquidity cushion**.
- **Croma Hardware Warranty Clearance:** Invokes the retail invoice parser to reconcile a Rs. 2,400 debit flagged at $z = 2.8\sigma$. Pulls invoice `#CR-49102`, proving purchase of a *Belkin HDMI 2.1 Ultra High Speed Cable* with an active **2-Year Manufacturer Warranty**, clearing the anomaly.

8 Dynamic Multi-Command Slack Sentinel

FinSight provides active monitoring by dispatching rich Slack Block Kit notifications to the `accounting` workspace (`backend/app/services/slack_service.py`).

8.1 Dual-Delivery Redundancy Architecture

Notifications are simultaneously delivered via two independent channels:

- **Public Channel Broadcast:** Dispatches rich alert cards to the public channel `#engineering-inventory-tracker` via Incoming Webhook for team auditability.
- **Private Bot Direct Message:** Posts to the user's personal bot DM (`@finsight`) via Bot User OAuth API (`chat.postMessage`).

8.2 Interactive Block Kit Message Schemas

Listing 2: Slack Block Kit Surcharge Warning Card Construction.

```
def build_swiggy_surge_card(order_data: Dict[str, Any]) -> List[Dict[str, Any]]:
    return [
        {
            "type": "header",
            "text": {"type": "plain_text", "text": "Swiggy Late-Night Surcharge Alert",
                "emoji": True}
        },
        {
            "type": "section",
            "fields": [
                {"type": "mrkdwn", "text": f"*Order ID:*\n{order_data['order_id']}"},
                {"type": "mrkdwn", "text": f"*Surge Markup:*\nRs. {order_data['surge_fee']}"},
                {"type": "mrkdwn", "text": f"*Restaurant:*\n{order_data['restaurant']}"},
                {"type": "mrkdwn", "text": f"*Hidden Fees:*\nRs.
                    {order_data['hidden_delivery_fees']}"}
            ]
        },
        {
            "type": "actions",
            "elements": [
                {"type": "button", "text": {"type": "plain_text", "text": "Set Dining Budget
                    Cap"}, "style": "primary", "value": "cap_dining"}
            ]
        }
    ]
```

9 Relational Schema & Data Integrity

The data layer uses SQLAlchemy 2.0 with Alembic migrations, running SQLite for local development and PostgreSQL for production [9].

9.1 PostgreSQL Conventions & Entity Architecture

- **Monetary Representation:** All monetary quantities use `BIGINT` integer minor currency units (`amount_minor`), where Rs. 1.00 = 100 paise. Inflow/credit is strictly positive (+); outflow/debit is strictly negative (−).
- **`User`:** Primary identity record storing monthly budget limits, savings targets, and voice persona preferences.
- **`Account`:** Represents financial accounts (checking, savings, credit cards) with masked identifiers and minor-unit balances.
- **`Transaction`:** Immutable ledger entries with `amount_minor` (integer paise), `category_id`, `payment_mode`, and timestamp.
- **`Budget`:** Monthly category spending targets with warning thresholds at 70%, 85%, and 100%.
- **`SavingsGoal`:** Target sums, accumulated savings, and priority ranks.
- **`RecurringObligation`:** Tracks subscriptions and utility dues with periodicity and autopay flags.
- **`ActionProposal`:** Audit log of pending and completed mutations (e.g., budget limit changes, bill authorizations) enforced with unique idempotency keys.

10 Frontend Architecture & UX Engineering

FinSight's client interface is constructed with React 19, TypeScript, and Tailwind CSS, adhering to strict spatial geometry specifications (1024 × 768 baseline, 12-column layout) [13].

10.1 Interactive Component Ecosystem

- **Sound Orb Visualizer (`SoundOrbVisualizer.tsx`):** A 3D animated canvas rendered with Framer Motion at 60 fps that reflects real-time audio amplitudes during both speech input and TTS playback. It features a three-layer gradient orb responsive to FFT frequency bins.
- **Live Browser Evidence Drawer (`AssistantDrawer.tsx`):** A sliding visual forensics drawer that displays headless browser screencasts, itemized vendor receipts, and verification badges.
- **Interactive Modular Cards:** Specialized chat cards including `SlackAlertCard` (providing deep links into Slack channels), `PortalEvidenceCard` (displaying extracted receipts), and `SimulationCard` (interactive what-if savings sliders).

11 Verification, Testing & Empirical Results

FinSight maintains an automated test suite consisting of 40 unit and integration tests, achieving a **100% pass rate** [12].

12 Competitive Benchmark & Production Roadmap

12.1 Competitive Analysis

Table 4 contrasts FinSight against leading commercial and research financial assistants.

Test Module	Pass / Total	Key Capabilities Verified
`test_api_endpoints.py`	11 / 11	REST route contracts, JSON schemas, auth middleware, pagination.
`test_data_integrity.py`	5 / 5	Foreign key integrity, double-entry balance reconciliations.
`test_ingestion.py`	2 / 2	Bank statement CSV normalization, merchant string parsing.
`test_money_math.py`	3 / 3	Paise integer arithmetic, float safety, currency conversion.
`test_operational_services.py`	5 / 5	Idempotent action proposals, two-phase commit confirmations.
`test_voice_generation_pipeline.py`	14 / 14	5-tier routing, BM25 retrieval, guardrail numeric stop-stream.
Total Test Suite	**40 / 40**	**100% Passing (Execution Time: 11.04 seconds)**

Table 3: Automated Verification and Test Suite Results.

Capability	**FinSight**	**Mint/YNAB**	**Revolut**	**ChatGPT+**	**Copilot**
Multilingual Voice AI (Hindi/En)	**Yes (Gnani)**	No	No	English Only	No
Sub-Second Voice Latency	**Yes (<800ms)**	N/A	N/A	Moderate	N/A
Zero-Hallucination SQL Grounding	**Yes (11 Tools)**	Yes	Yes	No	Partial
Headless Browser Forensics	**Yes (Playwright)**	No	No	No	No
Dynamic Slack Sentinel Cards	**Yes (Block Kit)**	No	No	No	No
Two-Phase Reversible Actions	**Yes**	No	No	No	No
Local Privacy-Preserving Mode	**Yes**	No	No	No	No

Table 4: FinSight Feature Benchmark Matrix against Existing Financial Systems.

12.2 Production Deployment Roadmap

1. **Account Aggregator (AA) Integration:** Evolve from CSV statement ingestion to real-time RBI-licensed Account Aggregator network synchronization.
2. **Direct BBPS Payment Switch:** Enable autonomous utility bill settlements through Bharat Bill Payment System (BBPS) upon explicit user biometric/voice confirmation.
3. **Distributed Worker Queue:** Deploy Celery / Redis Streams to decouple heavy browser scraping jobs from the primary FastAPI event loop [10].

13 Gnani.ai Strategic Alignment & Enterprise Business Model

A critical criterion for evaluating FinSight in the context of the **Gnani.ai Hackathon 2026** is its direct alignment with Gnani.ai's overarching commercial mission, product portfolio, and enterprise revenue model. Rather than functioning as a disconnected consumer prototype, FinSight is intentionally engineered as a flagship vertical solution showcasing the commercial power of Gnani.ai's speech intelligence stack within the high-value Banking, Financial Services, and Insurance (BFSI) sector.

13.1 Strategic Synergy with Gnani.ai's Mission

Gnani.ai is an established enterprise conversational and voice AI leader, trusted by tier-1 banks (e.g., ICICI Bank, SBI, Axis Bank) and insurance conglomerates to automate customer support, inbound inquiries, and debt collection. However, conversational banking faces a critical inflection point:

- **The Compliance & Hallucination Barrier:** Banks have historically hesitated to deploy LLMs for core financial advisory because generative models cannot guarantee arithmetic truth. A chatbot that invents an incorrect bank balance violates Reserve Bank of India (RBI) regulatory guidelines [3].
- **The FinSight Solution for Gnani:** FinSight provides the missing computational guarantee. By coupling Gnani's proprietary acoustic models with a deterministic SQL ledger and runtime numeric guardrails, FinSight gives Gnani.ai a production-ready, zero-hallucination reference architecture to pitch to BFSI chief technology officers (CTOs).

13.2 Direct Integration with the Gnani Product Suite

FinSight acts as a unified showcase for Gnani.ai's multi-product ecosystem:

1. **Gnani Prisma™ STT v3:** Powers sub-second streaming audio transcription over WebSockets, with dynamic code-switching (`indic-auto`, `en-IN`, `hi-IN`) for Indian banking jargon and colloquial Hindi.
2. **Gnani Timbre™ Neural TTS:** Delivers emotionally expressive, human-like voice synthesis with specialized personas: `Nalini`/`Deepak` for Hindi, `Kaveri`/`Pranav` for English, and `Poorvi` for Hinglish, applying customized prosody rules for Indian currency enunciation (e.g., naturally phrasing *"Rupees Forty-Five Thousand, Two Hundred"*).
3. **Armour365™ Voice Biometrics Integration (Roadmap):** FinSight's two-phase commit architecture is purpose-built to incorporate Gnani's Armour365 voice biometrics engine. Irreversible transactions (e.g., settling a utility bill or raising a budget ceiling) can require continuous, frictionless voiceprint authentication rather than clumsy SMS OTPs.

4. **Aura365™ Speech Analytics:** Aggregated voice session transcripts and sentiment metrics can be processed via Aura365 to deliver compliance audits, detect user financial stress, and surface cross-selling opportunities (e.g., pre-approved mutual funds when savings exceed targets).

13.3 Enterprise Business Model & Monetization

FinSight creates multiple high-margin enterprise revenue vectors for Gnani.ai:

- **1. B2B White-Label SaaS for BFSI:** Licensed to retail banks and neo-banks as a premium conversational intelligence layer inside mobile banking apps. Priced on an annual subscription per active customer.
- **2. High-Volume API Consumption Growth:** Every conversational turn, streaming audio chunk, and voice verification generates recurring consumption revenue across Gnani's Vachana, Timbre, and Armour365 APIs.
- **3. B2B Corporate Expense & Inventory Sentinel:** The Slack Sentinel integration extends FinSight into enterprise corporate expense auditing. Finance teams deploy FinSight in Slack to monitor corporate card burn rates and automatically verify receipt line items.
- **4. Financial Inclusion (Expanding into the "Bharat" Market):** Over 500 million Indian smartphone users reside in Tier-2 and Tier-3 cities where complex English banking dashboards create friction. By enabling natural voice interaction in regional Indian languages, FinSight allows Gnani's banking partners to acquire and retain non-English-speaking customers.

14 Hackathon Demonstration Scenarios & Transcripts

To validate real-world efficacy, FinSight was evaluated against three end-to-end multi-turn scenarios demonstrated live during the Hackathon.

14.1 Scenario A: Food Delivery Audit → Surge Fee Isolation → Slack Alert

User (English): *"Why was my dining spending so high this month?"*
FinSight: *"You spent Rs. 52,763 on Food & Dining, which is 191% of your budget limit. Swiggy alone accounted for Rs. 38,420 across 28 orders. Would you like me to inspect your latest late-night Swiggy order?"*
User (Hindi): *"Achha dikhao, receipt ko open karo!"*
FinSight (Action): [Launches Chromium sandbox → Scrapes Swiggy Order #SW-89218 → Slides open Evidence Drawer]
FinSight: *"Receipt verified. In Order #SW-89218 from Meghana Foods, your food subtotal was Rs. 1,050, but you were charged a Rs. 120 late-night surge fee and Rs. 65 packaging charge. Curbing post-11 PM orders could save Rs. 4,800 monthly. Should I send a surge fee alert to Slack?"*
User: *"Yes, send alert to Slack."*
FinSight (Action): [Dispatches interactive Block Kit card to #engineering-inventory-tracker and @finsight DM]
FinSight: *"Alert successfully posted to Slack with direct action buttons."*

14.2 Scenario B: Utility Verification → Liquidity Cushion

User: *"What bills are due soon?"*
FinSight: *"Your BESCOM electricity bill of Rs. 1,400 is due on February 28th. Would you like me to verify the meter reading on the portal?"*
User: *"Haan, verify kar do."*
FinSight (Action): [Navigates to BESCOM portal → Scrapes account #102938475]
FinSight: *"Verified on the BESCOM portal. Meter reading is 142 kWh for Rs. 1,400. Your checking account has Rs. 1,02,756, providing a 73x liquidity safety cushion. Autopay is scheduled for February 26th."*

14.3 Scenario C: Anomaly Clearance → Hardware Warranty Proof

User: *"Did I have any unusual transactions or anomalies this month?"*
FinSight: *"I flagged a Rs. 2,400 debit at Croma Retail on February 4th. It is a 2.8-sigma outlier relative to your normal electronics spend. Would you like me to inspect the official tax invoice on the portal?"*
User: *"Yes, open the Croma invoice."*
FinSight (Action): [Headless Chromium opens Croma tax invoice #CR-49102 → Extracts product specification]
FinSight: *"Verified. The transaction was for an authorized Belkin HDMI 2.1 Ultra High Speed Cable with an active 2-Year Manufacturer Warranty. This is a legitimate equipment purchase rather than a subscription leak. Should I post the clearance proof to Slack?"*
User: *"Haan, Slack par clearance bhej do."*
FinSight (Action): [Emits verified clearance Block Kit card to Slack]
FinSight: *"Audit clearance card posted to Slack. Anomaly flag has been safely resolved."*

15 Conclusion

FinSight establishes a new architectural standard for personal financial intelligence systems. By replacing speculative language model token generation with deterministic relational SQL grounding, bridging the visibility gap using Playwright headless browser forensics, broadcasting actionable insights via a dual-delivery Slack Sentinel, and delivering sub-second bilingual voice intelligence via Gnani AI Prisma and Timbre, FinSight transforms financial management from an opaque chore into an intuitive, verifiable, and autonomous co-pilot experience.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

[2] S. Robertson and H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[3] Reserve Bank of India. Guidelines on digital lending. *RBI/2022-23/111, DOR.CRE.REC.66/21.07.001/2022-23*, September 2022.

[4] National Payments Corporation of India. Unified Payments Interface (UPI) Procedural Guidelines. *NPCI Circulars and Operating Manuals*, 2024.

[5] Microsoft Playwright Contributors. Fast and reliable end-to-end testing and browser automation. *Playwright Architecture Documentation*, 2024.

[6] Gnani.ai Research. Sub-second neural speech synthesis and multilingual acoustic modeling for Indic languages. *Gnani.ai Technical Whitepaper Series*, 2025.

[7] FinSight Engineering. AI Layer Failure Analysis & Root Cause Post-Mortem. *FinSight Architecture Series*, 2026.

[8] FinSight Engineering. Voice Pipeline Architectural Upgrades & Hardware Caching Handoff. *FinSight Technical Documentation*, 2026.

[9] FinSight Engineering. Data Schema and Synthetic Data Specification. *FinSight Architecture Series*, 2026.

[10] FinSight Engineering. Production System Design and Capacity Planning. *FinSight Architecture Series*, 2026.

[11] FinSight Engineering. Technology Stack and Decision Principles. *FinSight Technical Documentation*, 2026.

[12] FinSight Engineering. Testing Strategy and Quality Plan. *FinSight QA Specification*, 2026.

[13] FinSight Engineering. Architectural Overrides and UI/UX Design Specifications. *FinSight Design Documentation*, 2026.